Fuzail Ayaz Khan
ENG24CY0102
3-B

Linux Programming Assignment 7

1. What is a bash shell script? Give one example. (CO4)

A bash shell script is a plain text file containing a sequence of commands that are executed by the bash shell interpreter. It serves as an automated way to perform tasks that would otherwise require manual input of multiple commands in a terminal. The script begins with a shebang line that specifies the interpreter path, followed by executable commands, variable assignments, control structures, and functions. Bash scripts are widely used for system administration, automation, file processing, and various computational tasks because they provide powerful text processing capabilities, conditional logic, loops, and integration with system utilities.

The structure of a bash script includes the shebang line at the beginning, which tells the system which interpreter to use for executing the script. Variables can be defined and used throughout the script, and the script can accept command-line arguments through special variables. Control structures like if-else statements, loops, and functions allow for complex logic implementation. Here is a simple example of a bash shell script:

```
#!/bin/bash
# This is a simple bash script example
echo "Welcome to Bash Scripting"
name="John"
echo "Hello, $name!"
current_date=$(date)
echo "Today's date is: $current_date"
```

This example demonstrates the basic components of a bash script including the shebang line, comments, variable assignment, command substitution, and output generation using the echo command.

2. Write a simple shell script to print "Hello World". (CO4)

A simple shell script to print "Hello World" is the traditional first program written when learning any programming language or scripting environment. In bash scripting, this involves creating a file with the appropriate shebang line and using the echo command

to display the desired text. The script file should be given executable permissions to allow it to run directly from the command line.

```
#!/bin/bash
echo "Hello World"
```

To make this script functional, you would save it to a file (for example, hello.sh), give it executable permissions using chmod +x hello.sh, and then run it with ./hello.sh. The shebang line #!/bin/bash ensures that the bash interpreter is used to execute the script, while the echo command outputs the text to the terminal. This simple example establishes the foundation for more complex bash scripting by demonstrating the basic structure and execution model.

3. What is the purpose of comments (#) in a shell script? (CO4)

Comments in shell scripts serve multiple crucial purposes that enhance code readability, maintainability, and collaborative development. The primary purpose is to provide human-readable explanations of code functionality, helping developers understand the logic, purpose, and context of specific commands or sections. Comments are particularly valuable when complex operations, regular expressions, or non-obvious logic are implemented, as they clarify the intent behind the code for future reference or for other developers who may need to maintain the script.

Comments also serve as documentation within the script itself, eliminating the need for separate documentation files for simple scripts. They can explain the overall purpose of the script, describe input parameters, document expected output, and provide usage examples. During development and debugging phases, comments are invaluable for temporarily disabling code sections without deleting them, allowing developers to test different approaches or isolate problematic sections. This commenting-out technique is particularly useful when troubleshooting or experimenting with alternative implementations.

Additionally, comments improve code maintenance by providing context about why certain decisions were made, what specific commands accomplish, and how different parts of the script interact. They can include information about dependencies, version requirements, known limitations, or potential security considerations. Well-commented

scripts are easier to modify, debug, and extend, making them more valuable for long-term use and team collaboration. Comments should be concise, relevant, and updated along with code changes to maintain their usefulness over time.

4. How do you declare variables (int, float, double, string, Boolean, and char) in a shell script? (CO4)

Variable declaration in bash shell scripting differs significantly from strongly-typed programming languages because bash treats all variables as strings by default, with type interpretation occurring during operations rather than at declaration time. Unlike languages such as C++ or Java, bash does not require explicit type declarations, and all variables are fundamentally stored as character strings regardless of their intended usage. However, bash provides mechanisms to influence how variables are treated and to achieve type-like behavior through various techniques and built-in commands.

For basic variable declarations, the syntax is simply variable_name=value without any type specifiers. String variables are declared as name="John Doe" or message='Hello World', with single quotes preserving literal values and double quotes allowing variable expansion. Integer-like behavior can be achieved using the declare command with the -i flag, such as declare -i number=42, which instructs bash to treat subsequent assignments to that variable as arithmetic expressions. Floating-point numbers require external tools like bc or awk for calculations since bash only supports integer arithmetic natively.

Boolean values in bash are typically represented using strings like true/false or integers like 1/0, with conditional tests determining their logical interpretation. Character variables are simply single-character strings, declared as char_var="A". Array variables can be declared using declare -a for indexed arrays or declare -A for associative arrays. While bash lacks traditional data types, the declare command provides options like -r for read-only variables, -x for exported variables, and -g for global scope variables, offering some level of variable attribute control similar to type systems in other languages.

5. Write a shell script to display the current date and time of the system. (CO4)

A shell script to display the current date and time utilizes the built-in date command, which provides extensive formatting options and timezone handling capabilities. The date command is a powerful utility that can display various date and time formats, perform date arithmetic, and work with different timezones. Creating a script for this purpose demonstrates basic output operations and command substitution techniques fundamental to bash scripting.

```
#!/bin/bash
# Script to display current date and time
echo "Current System Date and Time:"
echo "============================="
current_datetime=$(date)
echo "Standard format: $current_datetime"
echo "Custom format: $(date '+%Y-%m-%d %H:%M:%S')"
echo "Date only: $(date '+%A, %B %d, %Y')"
echo "Time only: $(date '+%I:%M:%S %p')"
echo "Unix timestamp: $(date +%s)"
```

This script demonstrates various ways to display date and time information, including the default format, custom formatting using format specifiers, and different representations like day names, month names, and timestamps. The date command's format specifiers allow for precise control over output appearance, making it suitable for logging, file naming, or user interface purposes. The script uses command substitution with $() to capture the date command output and store it in variables or display it directly.

6. Explain the difference between a constant and a variable in bash script. (CO4)

The distinction between constants and variables in bash scripting relates to mutability and the intended usage pattern throughout script execution. Variables in bash are mutable storage containers that can have their values changed multiple times during script execution through reassignment operations. They are declared simply by assignment without any special syntax, and their values can be modified, updated, or completely replaced as the script progresses. Variables provide the dynamic storage mechanism essential for scripts that process changing data, user input, or computational results.

Constants, while not having a dedicated keyword in bash like some programming languages, are typically implemented using the readonly command or declare -r option to create immutable values that cannot be changed after initial assignment. Once a variable is marked as readonly, any attempt to modify its value results in an error, effectively creating constant behavior. Constants are used for values that should remain fixed throughout script execution, such as configuration parameters, mathematical constants, file paths, or system-specific values that should not be accidentally modified.

The practical difference lies in their intended usage patterns and error prevention capabilities. Variables are suitable for counters, temporary storage, user input, and any data that naturally changes during script execution. Constants provide protection against accidental modification of critical values and make script intent clearer by explicitly marking which values should remain unchanged. Best practices suggest using descriptive naming conventions like uppercase letters for constants and lowercase for variables, though bash itself does not enforce these conventions. Constants also serve as a form of documentation, indicating to other developers which values are fundamental to the script's operation and should not be altered.

7. Write a shell script to read two integer number from the user and compute the sum of both the number. (CO4)

A shell script that reads two integers from user input and computes their sum demonstrates fundamental concepts of user interaction, input validation, and arithmetic operations in bash scripting. This type of script showcases the use of the read command for user input, variable assignment, and arithmetic expansion for mathematical calculations. The script should handle user prompts clearly and provide meaningful output that shows both the input values and the calculated result.

```
#!/bin/bash
# Script to read two integers and compute their sum
echo "Integer Addition Calculator"
echo "==========================="

# Read first integer
echo -n "Enter the first integer: "
read num1
```

```
# Read second integer
echo -n "Enter the second integer: "
read num2

# Compute the sum using arithmetic expansion
sum=$((num1 + num2))

# Display the result
echo ""
echo "Results:"
echo "First number: $num1"
echo "Second number: $num2"
echo "Sum: $num1 + $num2 = $sum"
```

This script uses the read command to capture user input and store it in variables, then performs arithmetic addition using the $((expression)) syntax, which is bash's built-in arithmetic expansion mechanism. The script provides clear prompts and formatted output to enhance user experience. For more robust implementations, input validation could be added to ensure that users enter valid integers, and error handling could manage cases where non-numeric input is provided.

8. What is the use of source command in shell scripting? (CO4)

The source command in shell scripting serves the critical purpose of executing commands from a specified file within the current shell environment rather than creating a new subshell process. This fundamental distinction makes source essential for scenarios where script execution needs to affect the current shell's environment, including variables, functions, aliases, and configuration settings. When a script is executed normally, it runs in a subshell, and any environment changes are lost when the subshell terminates. However, source ensures that all changes persist in the calling shell environment.

The primary applications of the source command include loading configuration files, importing function libraries, setting environment variables, and applying profile changes without requiring a new shell session. For example, when you modify your .bashrc file and want the changes to take effect immediately, you use source ~/.bashrc rather than logging out and back in. The command is also essential for creating modular

scripts where common functions are stored in separate files and sourced into multiple scripts, promoting code reusability and maintainability.

The source command accepts arguments that are passed as positional parameters to the sourced file, enabling parameterized execution of configuration scripts or function libraries. It searches for files in the directories specified by the PATH environment variable if a relative path is provided, or it looks in the current directory if the file is not found in PATH. The exit status of source reflects the success or failure of the sourced file's execution, allowing error handling in the calling script. This command is synonymous with the dot (.) operator, which provides the same functionality with shorter syntax.

9. How can you debug a shell script? Give two methods. (CO4)

Debugging shell scripts requires systematic approaches to identify and resolve errors, logic problems, or unexpected behavior in script execution. The two most effective and commonly used debugging methods in bash scripting are the debug mode execution and strategic use of echo statements for runtime inspection. These methods provide different levels of detail and serve complementary purposes in the debugging process, allowing developers to trace execution flow, examine variable values, and identify problematic sections of code.

The first method involves using bash's built-in debug mode by executing the script with the -x option or by adding set -x within the script itself. This debug mode causes bash to print each command along with its expanded arguments before execution, providing a complete trace of the script's execution path. Running bash -x scriptname.sh or adding set -x at the beginning of the script enables this comprehensive debugging output. For selective debugging, you can use set -x to enable debugging for specific sections and set +x to disable it, allowing focused analysis of problematic code areas without overwhelming output from the entire script.

The second method involves strategically placing echo statements throughout the script to display variable values, execution checkpoints, and intermediate results. This approach allows precise control over what information is displayed and when, making it particularly useful for examining variable contents, verifying conditional logic, and tracking program flow through complex sections. Echo statements can display variable

values, indicate which branches of conditional statements are executed, and provide markers showing script progress. This method is especially valuable when combined with conditional debugging, where echo statements are controlled by a debug flag variable, allowing debugging output to be easily enabled or disabled without removing the debugging code.

10. Write a bash script to create and delete a file. (CO4)

A bash script that creates and deletes a file demonstrates fundamental file manipulation operations and provides a foundation for more complex file management tasks. This type of script showcases the use of file creation commands, existence checking, and file removal operations while incorporating proper error handling and user feedback. The script should handle various scenarios including file existence checking, permission issues, and providing informative output about the operations performed.

```
#!/bin/bash
# Script to create and delete a file
filename="test_file.txt"

echo "File Management Script"
echo "======================"
echo "Working with file: $filename"
echo ""

# Create the file
echo "Creating file: $filename"
if touch "$filename"; then
    echo "√ File created successfully"

    # Add some content to the file
    echo "This is a test file created by bash script" > "$filename"
    echo "√ Content added to file"

    # Display file information
    echo "File information:"
    ls -l "$filename"
    echo ""

    # Pause before deletion
    echo "Press Enter to delete the file..."
```

```
    read

    # Delete the file
    echo "Deleting file: $filename"
    if rm "$filename"; then
        echo "✓ File deleted successfully"
    else
        echo "✗ Error: Failed to delete file"
        exit 1
    fi
else
    echo "✗ Error: Failed to create file"
    exit 1
fi

echo "Script completed successfully"
```

This comprehensive script demonstrates file creation using the touch command, content addition using output redirection, file information display using ls, and file deletion using rm. It includes error handling to check the success of each operation and provides clear feedback to the user about each step. The script also includes a pause before deletion to allow the user to examine the created file, making it educational and interactive for learning purposes.